Learn Python

Teacher Edition

Minute-by-minute session playbooks · transition scripts · predicted misconceptions · Socratic prompts

A 5-session, ~12-hour (two hours per session, + capstone) accelerated Python course for a PhD-in-Education learner with no prior coding experience.

Generated 2026-06-29 · Instructional content original

Learn Python — TEACHER Edition

Everything in the student syllabus, **plus** the instructor scaffolding: a minute-by-minute clock for each two-hour session, transition scripts (how to move between blocks without dead air), predicted misconceptions for *this* learner, Socratic prompts (DeepTutor-style — ask, don't tell), and an explicit **"if you're behind, cut this"** line per session.

Each two-hour session covers **two topics** — **Part A** and **Part B** — around a mid-session break. That's the point of the format: a fast learner clears a topic in under an hour, so we pair two related topics per session instead of stretching one thin.

How to read this document

Each session has:

- **Covers** — the two topics (Part A / Part B) this session bundles.
- **Pre-flight** — what to have open/ready before the student arrives.
- **The clock** — a 120-minute breakdown: Part A, a break, Part B, then a combined recap. Keep a timer visible. The numbers are targets, not law.
- **Transitions** — short scripted lines to hand off cleanly between blocks.
- **Predicted misconceptions** — where THIS learner (expert in research, novice in code) will stumble.
- **Socratic prompts** — questions to ask instead of explaining; let them derive it.
- **Cut line** — the first thing to drop if you're running over.

Universal pacing principles (this learner is fast)

- **Talk less than you want to.** He reads fast and abstracts well. Default to "here's the rule, here's the trap, now you try."
- **Protect — and fill — the practice time.** Roughly **an hour of hands-on** per session, split across the two halves plus a short combined block. It's packed on purpose: he finishes individual tasks quickly, so each half carries *more* tasks rather than more minutes. If concept overruns, steal from your own talking, never from his typing.
- **Predict-then-run is the engine, especially the Session 1 traps.** Always have him *commit to an answer out loud* before running. The cognitive surprise is the teaching moment.
- **Two topics, one through-line.** Each pairing shares a thread (types→the type traps; functions→recursion; exceptions→dirty file data). Name the thread at the Part A→B handoff so the session feels like one arc, not two.
- **Don't pad.** Each clock is tight by design. If he's ahead, give him the next practice task or a stretch goal, not more talking.
- **Carry a running "misconceptions log"** (DeepTutor "learning memory"): note every trap he hit this session; re-surface it as a 60-second warm-up next session.

SESSION 1 — Running Python, Types & the Type Traps

Covers: Part A — Running Python, Variables & Types; Part B — the Dynamic-Typing Traps (the most important material in the course). **Pre-flight:** terminal + VS Code open; `examples/session-01/` ready; a deliberately broken line staged to show a traceback; `cheatsheets/traps-and-gotchas.md` open for Part B.

The clock (120 min)

- **0:00–0:06 — Orientation.** Why Python, why this re-ordered path, how a two-hour session works (two halves, one break). Don't oversell; he wants to start.
- **0:06–0:26 — Part A concept.** REPL vs script; `python file.py`. Variables as *labels on objects* (Connection Map #1). The five core types; `type(x)`. `input()` → always `str`. f-strings. `int()/float()/str()`.
- **0:26–0:48 — Part A live + you-try.** Build `greet.py` then "years to graduation" together; trigger and *read* one traceback (last line first). Then he starts `examples/session-01/practice.md` Part A (GPA reporter, age bucket, the string-concatenation trap).
- **0:48–0:56 — Break.**
- **0:56–1:22 — Part B concept + demo (predict-then-run).** This is the heart of the course; tell him so. `==` vs `is` (value vs identity, Connection Map #3) with a mutable-list demo and `is None`; `bool ⊂ int` (`True == 1`, `sum([T,F,T])`, Connection Map #4); **float precision** `0.1 + 0.2` → `math.isclose` (Connection Map #5); `5 == "5"` is `False` but `5 > "5"` raises; `isinstance` vs `type`; truthiness. He predicts every line *out loud* before you run it.
- **1:22–1:48 — Part B you-try.** `examples/session-01/practice.md` Part B: the predict-the-output gauntlet, then `clean_score()` handling int/float/str (and rejecting `bool`).
- **1:48–1:56 — Traps recap (both halves).** `input()` → string; `int(3.9)` truncates; `==` vs `is`; never `==` raw floats. Hand him the trap cheat sheet as his permanent reference.
- **1:56–2:00 — Summary + quiz. Next:** control flow & data structures.

Transitions

- A concept→live: *"Enough theory — watch me make these mistakes so you don't have to."*
- A→B (the thread): *"You now know the five types. Here's the catch: Python lets them mix in ways that surprise everyone. Cover the right column and predict each line."*
- B→recap: *"These eight traps are 80% of week-one bugs. Let's name them once more, then they're yours."*

Predicted misconceptions (this learner)

- Expects `input()` to give a number → `"5" + "3" == "53"`.
- Thinks `=` asserts equality (math habit). Reinforce label-on-object now; it pays off in Part B and in Session 2's aliasing.
- Assumes `is` is a stylistic `==` — nail it with the mutable-list demo.
- Trusts `==` on floats out of stats habit ("I round anyway") — stress the *cause* is binary storage, not data.

- Over-generalizes the `TypeError` and thinks `5 == "5"` errors too. It returns `False` — show it.

Socratic prompts

- "What type do you think `input()` hands back? How could we check?" (→ `type(...)`)
- "Two students, same 3.7 GPA. `==`? `is`? Why?"
- "If `0.1 + 0.2` isn't `0.3`, is the bug in the data or in the computer? How would you test 'close enough'?"
- "`5 == '5'` is False, fine. So why does `5 > '5'` *crash*?"

Cut line: drop the small-int-cache and `nan` wrinkles, and the years-to-graduation demo; **never** cut the Part B `==` / `is`, float, and `isinstance` core or its practice.

SESSION 2 — Control Flow & Data Structures

Covers: Part A — Conditionals & Loops; Part B — list / tuple / dict / set. **Pre-flight:** `examples/session-02/`; a messy nested-`if` staged for the refactor; `Ctrl+C` ready for the infinite-loop demo; a "list of dicts = tidy dataset" diagram (Connection Map #6).

The clock (120 min)

- **0:00–0:06 — Warm-up.** S1 traps recap (60-sec quiz from your log).
- **0:06–0:30 — Part A concept.** `if/elif/else`; **chained comparisons** (`90 <= x < 100` — reads like math); `and/or/not`, short-circuit, `and/or` return an *operand*; `while` (+ infinite-loop demo); `for ... in`; `range` (off-by-one!); `break` / `continue`; the `while True: … break` validation pattern; **`enumerate` / `zip`** as the antidote to `range(len(...))`.
- **0:30–0:52 — Part A live + you-try.** A Likert→label classifier (chained comparison + early `return`); sum/average a roster two ways (index vs `enumerate` / `zip`); a robust "ask until valid" loop. He then starts `examples/session-02/practice.md` Part A (grade-band classifier — test every boundary — and the validation loop).
- **0:52–1:00 — Break.**
- **1:00–1:24 — Part B concept + live.** `list` (mutable) / `tuple` (immutable) / `dict` (key→value) / `set` (unique); indexing & **slicing**; nesting → **list of dicts = a dataset**; list/dict comprehensions; `sorted(key=lambda …)`. Live: build a roster as a list of dicts, sort by score, dedupe answers with a `set`, rewrite a loop as a comprehension.
- **1:24–1:48 — Part B you-try.** `examples/session-02/practice.md` Part B: group by grade band into a dict, `{name: average}` in one comprehension, slice/sort tasks, and the **aliasing** bug then its fix.
- **1:48–1:56 — Traps recap (both halves).** `if x == True` (just `if x`); `range(1,5)` excludes 5; **mutating a list while looping it**; `range(len(...))` vs `enumerate` / `zip`; **aliasing** (`b = a` shares the list) and `[[0]*3]*3` shared rows.
- **1:56–2:00 — Summary + quiz. Next:** functions, scope & recursion.

Transitions

- A concept→live: *"Watch me turn an ugly five-level `if` into three readable lines — then loop a whole roster."*
- A→B (the thread): *"You can loop over data now. Next: the containers worth looping — and a list of dicts is just a tidy dataset, rows as dicts, keys as your variables."*
- B→recap: *"One last thing that bites everyone — assignment doesn't copy. Watch `b` change when I never touched it."*

Predicted misconceptions

- Writes `if score >= 90 and score < 100` — show chaining `90 <= score < 100`.
- Boundary errors (`>=` vs `>`) at cutoffs — make him test 89.999 / 90 / 90.001.
- `range(len(x))` index habit (from R/SPSS vectorized thinking) — push `enumerate` / `zip`.
- Removes items from a list *while iterating it* → demo the bug, then iterate a copy / build a new list.

- **Aliasing** is the big one — expects `b = a` to copy. Show `a.append(...)` changing `b`; tie to S1 "label on object."

Socratic prompts

- "`x = 5 and 0` — what's `x`? Why isn't it `True`/`False`?"
- "You need both the position and the value. What's cleaner than indexing?"
- "`b = a; a.append(99)` — what's in `b`? Why? (Labels, not boxes.)"
- "Survey gave duplicate free-text answers. What structure removes duplicates for free?"

Cut line: drop `match/case`, `for/else`, and deep-copy/`[[0]*3]*3`; keep chained comparisons, `enumerate`/`zip`, the validation loop, comprehensions, and aliasing basics.

SESSION 3 — Functions, Scope & Recursion

Covers: Part A — Functions, Scope & Reusability; Part B — Recursion. **Pre-flight:** `examples/session-03/`; the **mutable-default-arg** demo staged (Part A headline); a nested-JSON-shaped dict staged for `deep_sum`, and the `runaway` overflow + `sys.getrecursionlimit()` queued for Part B.

The clock (120 min)

- **0:00–0:06 — Warm-up.** S2 traps recap (aliasing, mutate-while-iterating).
- **0:06–0:28 — Part A concept.** `def`; parameters (positional/keyword/default, `*args` / `**kwargs`); `return` vs `print`; scope (LEGB), `global` and why to avoid it; docstrings; **type hints** ("not enforced; `mypy` checks them"). Keyword-only args and a one-line decorator if time allows.
- **0:28–0:52 — Part A live + you-try.** Refactor S2 inline code into `class_average` / `letter_grade`; then the **mutable-default bug** live (`def add(x, bag=[])` accumulating) → fix with `bag=None`. He starts `examples/session-03/practice.md` Part A (a small grade-functions library with hints/docstrings; reproduce-then-fix the mutable default).
- **0:52–1:00 — Break.**
- **1:00–1:24 — Part B concept + live.** The two parts: a **base case** and a **recursive case** that moves toward it. Trace `factorial(3)` as a stack that builds then unwinds; recursion vs iteration; the cost (each pending call is a frame; **no tail-call optimization**, limit ≈ 1000). Live: `countdown` / `factorial`, then `deep_sum` over nested data (the payoff a single loop can't reach), then a `RecursionError` read together. Mention `@lru_cache`.
- **1:24–1:48 — Part B you-try.** `examples/session-03/practice.md` Part B: recursive sum, `flatten`, `depth`, and the two trap-fixes (missing base case; forgetting to `return` the recursive call).
- **1:48–1:56 — Traps recap (both halves).** Mutable default; forgetting `return` (function/ recursion returns `None`); `UnboundLocalError`; unreachable base case → stack overflow; recursion isn't free.
- **1:56–2:00 — Summary + quiz. Next:** exceptions, files & research data.

Transitions

- A concept→live: *"This next bug has burned every Python programmer once. Watch the list keep growing across calls."*
- A→B (the thread): *"A function can call other functions. The mind-bender: it can call* itself*. That's exactly the tool for data defined in terms of itself — nested data."*
- B live→you-try: *"Say the base case out loud before you write the function — that's where the bugs hide."*

Predicted misconceptions

- Won't believe the default list persists until shown twice; then explain defaults evaluate *once, at definition*.
- Thinks a function that `print`s has "returned" the value → show `x = show(...)` is `None`.
- Expects type hints to enforce types → show `add("a","b")` still runs.

- Writes the recursive case but forgets to `return` it → silent `None`.
- From a vectorized stats background, may not see when recursion beats a loop → the nested-data demo is the "aha"; and may assume recursion is a free swap → show the limit.

Socratic prompts

- "I called `add(1)` three times and the list keeps growing. When does `bag=[]` actually run?"
- "`print` vs `return` — which lets the *next* function use the result?"
- "What's the smallest input where the answer is obvious without recursing? That's your base case."
- "Your data is a list that can contain lists. What kind of function matches a thing defined in terms of itself?"

Cut line: drop decorators/closures in Part A and the `depth`/string-reverse tasks in Part B; **never** cut the mutable-default demo or `deep_sum` on nested data.

SESSION 4 — Exceptions, Files & Research Data

Covers: Part A — Exceptions & Defensive Code; Part B — Files, Libraries & Research Data. **Pre-flight:** `examples/session-04/` with `students.csv` + `survey.csv`; a dirty list ("N/A", "", "7" on a 1–5 scale) staged; confirm `pip` works and have `pandas` ready for the teaser.

The clock (120 min)

- **0:00–0:06 — Warm-up.** S3 traps recap (mutable default; forgotten `return`).
- **0:06–0:28 — Part A concept.** Errors vs exceptions; `try/except/else/finally`; common types (`ValueError`, `KeyError`, `FileNotFoundError`); `raise ValueError(...)`; a custom exception subclass; **EAFP** vs LBYL; `assert` (a developer check, can be disabled — not input validation).
- **0:28–0:52 — Part A live + you-try.** Harden `safe_int()` / `clean_likert()` against blanks/"N/A"/out-of-range; a first `pytest.raises` test. He starts `examples/session-04/practice.md` Part A (validate a raw response list into clean values + a rejection log; add a `raise`).
- **0:52–1:00 — Break.**
- **1:00–1:24 — Part B concept + live.** `open` / `with` (auto-close); modes (`r/w/a`; `w` **overwrites!**); **CSV** via `csv.DictReader` / `DictWriter` (rows as dicts → ties to S2); `json`; researcher stdlib (`statistics`, `datetime`, `pathlib`). Live: read `students.csv` → list of dicts, class mean with `statistics.mean`, write a summary CSV; then the **`pandas` teaser** ("same thing in three lines — that's your next course").
- **1:24–1:48 — Part B you-try.** `examples/session-04/practice.md` Part B: per-item survey means skipping dirty values, write `survey_summary.csv`, mean score by major; the exception skills from Part A do the cleaning.
- **1:48–1:56 — Traps recap (both halves).** Bare `except:`; swallowing errors; `"w"` destroys data; forgotten `newline=""`; reading a file twice (cursor at end).
- **1:56–2:00 — Summary + quiz. Next:** regular expressions, modules & OOP.

Transitions

- A concept→live: *"Your real survey data WILL have 'N/A' in a numeric column. Let's write code that shrugs it off."*
- A→B (the thread): *"You can now survive one bad value. Real data is a* file full *of them — let's open a real CSV and clean it with exactly these moves."*
- Teaser framing: *"Everything you just did by hand, pandas does in three lines — your next course, not today's. Now you know what it's doing underneath."*

Predicted misconceptions

- Reaches for `if/else` (LBYL) everywhere; introduce EAFP as the Pythonic default, but be honest both are valid.
- Writes bare `except:` — show why `except ValueError:` is safer.
- Confuses `raise` with `return`, and `assert` with input validation.
- Opens with `"w"` to read and wonders why the file is empty; stress mode meanings.

- Expects to iterate a file object twice without re-opening; show the exhausted cursor.

Socratic prompts

- "User typed 'seven' instead of 7. Catch it *before* (`if`) or *after* (`try`)? Trade-offs?"
- "Why is a bare `except:` dangerous? What might you swallow?"
- "Why does `with` matter even if your program crashes? What does it guarantee?"

Cut line: drop the custom exception and `pytest` (use `assert`) in Part A, and the `pandas`/`json` teaser in Part B; keep `try/except` validation and `csv.DictReader/Writer`.

SESSION 5 — Regular Expressions, Modules & OOP

Covers: Part A — Regular Expressions & Text Cleaning; Part B — Modules, OOP & the Pythonic Toolkit. **Pre-flight:** `examples/session-05/`; messy free-text, emails, and "Last, First" names staged for Part A; `grades.py` + the `Student` class and the generator-exhaustion demo staged for Part B; regex101 open.

The clock (120 min)

- **0:00–0:06 — Warm-up.** S4 traps recap (bare `except:`; `"w"` overwrites).
- **0:06–0:28 — Part A concept.** Why regex for a researcher (validate/extract/clean/qualitative-coding, Connection Map #9). **Raw strings** `r"..."`. Survival tokens (`. \d \w \s + * ? {m,n} ^ $ [] () |`); `.` matches *any* char (use `\.`). The four functions: `search` / `fullmatch` / `findall` / `sub`; capture groups (and named groups).
- **0:28–0:52 — Part A live + you-try.** Validate an email (`fullmatch`), extract dept+number with groups, collapse whitespace with `sub`, count `#hashtags` with `findall`. He starts `examples/session-05/practice.md` Part A (email validator, extract codes, hashtag count, the `"Last, First"` flip, and one case where `.split()` beats regex).
- **0:52–1:00 — Break.**
- **1:00–1:24 — Part B concept + live.** Modules: move grade functions to `grades.py`, `import`, the `if __name__ == "__main__":` guard. OOP: a small `Student` class — `__init__`, `self`, a method, `__str__`, a validating `@property` setter (Connection Map #10), brief inheritance with `super()`; `@dataclass` for the free `__repr__` / `__eq__`.
- **1:24–1:48 — Part B you-try.** `examples/session-05/practice.md` Part B: build the validating `Student`, add `GradStudent(super())`, then a tour-by-doing of the Pythonic toolkit (comprehension, `map` / `filter`, a generator + its one-pass exhaustion, the walrus `:=`).
- **1:48–1:56 — Traps recap (both halves).** Forgot `r"..."`; `re.search` returns `None` → guard before `.group()`; `self` confusion; a generator exhausts after one pass; over-using a class where a dict/function fits.
- **1:56–2:00 — Summary + quiz; course wrap. Next:** the capstone project.

Transitions

- A concept→live: *"Four functions cover almost everything: search, fullmatch, findall, sub. Every pattern is a raw string, no exceptions."*
- A→B (the thread): *"You can now clean any string. Last step: organize your code so it's reusable — functions into modules, then data-plus-behavior into a class."*
- Modules→OOP: *"Functions in a file is reuse. A class bundles the data* and *the rules that guard it."*

Predicted misconceptions

- Forgets raw strings → backslash chaos. Always `r"..."`.
- Calls `.group()` on a `None` result → teach the `if m:` guard.
- `self` looks magical — it's just "this particular instance."
- Treats a generator like a list (iterates once) → show the second pass is empty.

- Reaches for a class when a function or dict would do — name when OOP earns its keep.

Socratic prompts

- "You want every response mentioning a theme. Is that a *form* match (regex) or a *meaning* match (human coding)? What can regex actually catch?"
- "Why does `re.search(...).group()` crash when there's no match? (Same shape as `5 > '5'` weeks ago.)"
- "Your operational definition of 'student' — what attributes and rules belong to it? That's your class."
- "A million rows won't fit in memory. What does `yield` give you that a list doesn't?"

Cut line: drop named groups / `re.VERBOSE` in Part A and `@dataclass` / `match` - `case` in Part B; compress the toolkit tour to comprehensions + `enumerate` / `zip`. Keep validate+extract+`sub`, modules, and the validating class.

SESSION 6 (Optional) — Capstone

Role shift: you stop teaching and start *coaching*. He drives; you ask questions and unblock.

- **0:00–0:15 — Brief & plan.** He restates the goal and sketches the steps aloud (pseudocode). You only check the plan is sound.
- **0:15–1:05 — Build.** He codes the Gradebook & Survey Analyzer (`assessments/capstone-project.md`). Intervene only when stuck >3 min; prefer a question over an answer.
- **1:05–1:13 — Break.**
- **1:13–1:45 — Build, continued.** Finish the report CSV, then one stretch goal (a `Student` class, a regex validation, or the recursive nested-data total).
- **1:45–1:55 — Review.** Walk his code for the course's traps (identity, aliasing, mutable defaults, bare `except:`). Praise readability.
- **1:55–2:00 — Debrief & next steps.** Point to pandas/visualization as the genuine next course.

Coaching prompts: "What's your data structure?" · "What happens if that cell is blank?" · "Is that comparing value or identity?" · "Could that be one comprehension?"

Instructor's running checklist (use across all sessions)

- [ ] Timer visible; both halves get real hands-on time; the break is honored.
- [ ] The Part A→B thread named out loud, so two topics feel like one arc.
- [ ] Misconceptions log updated after each session; warm-up next session pulls from it.
- [ ] Every trap demoed via **predict-then-run**, not narration.
- [ ] Each new concept hooked to the **Connection Map** before syntax.
- [ ] Student typed everything himself; you talked less than half the session.
- [ ] End-of-session quiz given; capstone kept in view as the destination.

Appendix A — Master Course Outline

Master Course Outline — Learn Python

> Single source of truth. The student and teacher syllabi both derive from this. **5 core two-hour sessions** + 1 optional capstone session (~12 hours core, ~14 with the capstone). Each session covers **two paired topics** (Part A + Part B) around a mid-session break.

How the topics are sequenced

A typical intro-Python course teaches roughly: functions/variables → conditionals → loops → exceptions → libraries → unit tests → file I/O → regular expressions → OOP → assorted "power-tools."

We **re-sequence and pair** for a fast adult learner. A strong self-learner clears one of those topics in well under an hour, so stretching one across a two-hour session wastes the back half. Instead each two-hour session bundles **two related topics** that share a through-line:

1. **Types → the type traps.** You can't reason about the traps until you know the types, and the traps are the whole reason this course exists — so they lead, together, on day one.
2. **Control flow → data structures.** Loops are most useful over the containers worth looping; a list of dicts is just a dataset.
3. **Functions → recursion.** Recursion is a function that calls itself, so it lands while function mechanics are fresh — and nested data (from S2) is the payoff.
4. **Exceptions → files & research data.** Surviving one dirty value scales straight into cleaning a whole CSV of them.
5. **Regex → modules & OOP.** The two "finishing" skills: clean any text, then organize code into modules and a small class.

The power-tools (comprehensions, generators, `*args`, type hints, the walrus) are folded into the sessions where they naturally belong, not saved for the end.

Design rules (from the e-learning pipeline)

- **Every session = two hours = two paired topics**, run **Part A → break → Part B → combined recap**.
- Each half is **Concept → Live Example → Practice**; practice is woven into both halves plus a short combined block (**~1 hour hands-on total**), packed because this learner moves fast.
- Every session has 4–6 learning objectives (≈2–3 per half); every objective is testable in that session's quiz.
- Every abstract idea ships with a runnable, education-flavored example.
- Difficulty rises monotonically; nothing is used before it's introduced (except clearly-flagged teasers).

Session 1 — Running Python, Types & the Type Traps

Part A — Running Python, Variables & Types · Part B — the Dynamic-Typing Traps (the core of the course)

Objectives

1. Run Python interactively (REPL) and as a `.py` script; read a traceback without panic.
2. Use the core types (`int`, `float`, `str`, `bool`, `None`); do I/O with `input()` / `print()`, f-strings, and `int()` / `float()` / `str()` (remember `input()` is always a `str`).
3. Distinguish **value equality (`==`)** from **identity (`is`)**; predict `bool ⊂ int` (`True == 1`), `int` / `float`, and **float precision** (`0.1 + 0.2`).
4. Check types with `isinstance()` vs `type()`, handle `5 == "5"` vs `5 > "5"`, and reason about truthiness.

Why it leads: the whole course is built around the type traps (`==` vs `is`, `True == 1`, `0.1 + 0.2`, `5 == "5"`). Front-loading them — right after the types they concern — means every later session can assume the fluency.

Session 2 — Control Flow & Data Structures

Part A — Conditionals & Loops · Part B — list / tuple / dict / set

Objectives

1. Write `if` / `elif` / `else` with comparison & logical operators and **chained comparisons**; use `and` / `or` / `not` correctly (short-circuit, operand-return); avoid `if x == True`.
2. Write `for` / `while` loops; control them with `break` / `continue`; mind `range` off-by-one; iterate Pythonically with `enumerate` / `zip` and the `while True:` validation loop.
3. Choose between `list` / `tuple` / `dict` / `set`; index, slice, and nest them; build list/dict **comprehensions**; sort with `sorted(key=…)`.
4. Reason about **mutability & aliasing** (copy vs reference, `[[0]*3]*3` shared rows).

Trap focus: `if x == True`, `range(1,5)` excludes 5, mutating a list while iterating it, `range(len(...))` instead of `enumerate` / `zip`, and **aliasing** (`b = a` shares the list).

Session 3 — Functions, Scope & Recursion

Part A — Functions, Scope & Reusability · Part B — Recursion

Objectives

1. Define functions with positional, keyword, default, `*args`, and `**kwargs` parameters.
2. Explain scope (LEGB) and `return` vs `print`; document with docstrings and **type hints** (and know hints aren't enforced); dodge the `UnboundLocalError` / `global` trap.
3. Write a recursive function with a correct **base case** and **recursive case**; trace the **call stack**; convert between recursion and iteration; know recursion's cost (`RecursionError`, no tail-call optimization, limit ≈ 1000).
4. Apply recursion to **naturally nested data** (nested lists/dicts/JSON) where one loop is awkward.

Trap focus: the **mutable default argument** bug; forgetting to `return` (function or recursive call returns `None`); an unreachable base case → stack overflow.

Session 4 — Exceptions, Files & Research Data

Part A — Exceptions & Defensive Code · Part B — Files, Libraries & Research Data

Objectives

1. Handle errors with `try` / `except` / `else` / `finally`; `raise` deliberately; validate messy human/research input the EAFP way; use `assert` and write a first `pytest` test.
2. Read/write text with `open` / `with` and understand file modes (why `"w"` is dangerous).
3. Load and write **CSV** survey/gradebook data with `csv.DictReader` / `DictWriter`; touch `json`.
4. `import` the researcher's stdlib (`statistics`, `datetime`, `pathlib`), `pip install` a package, and meet `pandas` in a guided teaser.

Trap focus: bare `except:`, swallowing errors, `"w"` silently overwrites, forgotten `newline=""`, reading a file twice.

Session 5 — Regular Expressions, Modules & OOP

Part A — Regular Expressions & Text Cleaning · Part B — Modules, OOP & the Pythonic Toolkit

Objectives

1. Write patterns with raw strings and the core tokens; use `re.search` / `fullmatch` / `findall` / `sub` to validate, extract (capture groups), and clean real research text.
2. Split code into modules and `import` them; understand the `if __name__ == "__main__":` guard.
3. Model a domain entity with a small **class** (`__init__`, `self`, `__str__`, a validating `@property`, brief inheritance with `super()`).
4. Apply the **Pythonic toolkit**: comprehensions, `map` / `filter`, `enumerate` / `zip`, generators/ `yield`, the walrus `:=`.

Trap focus: forgetting `r"..."`, `.` matches any char, `re.search` returns `None`, `self` confusion, a generator exhausts after one pass, over-using a class where a function/dict fits.

Session 6 (Optional) — Capstone Project *(integrative)*

Objective: independently build one small, end-to-end program on a real-ish education dataset. Default brief: **"Gradebook & Survey Analyzer"** — read a CSV of students + Likert responses, clean and validate it, compute summary statistics, flag at-risk students, and write a report CSV. Alternative briefs are listed in `assessments/capstone-project.md`.

Coverage check (every core topic lands somewhere)

Topic	Where it lives now
Running Python, variables & types	S1 Part A
The dynamic-typing traps (`==` / `is`, floats, `bool⊂int`, `isinstance`)	S1 Part B
Conditionals	S2 Part A
Loops	S2 Part A (iterating data in S2 Part B)
Data structures (list/tuple/dict/set)	S2 Part B
Functions, scope & reuse	S3 Part A
Recursion	S3 Part B (nested data ties to S2 Part B)
Exceptions & unit tests	S4 Part A
Files & libraries (CSV, `statistics`, `pandas` teaser)	S4 Part B
Regular expressions	S5 Part A
Modules & OOP	S5 Part B
Power-tools (comprehensions, `*args`, type hints, generators, `map` / `filter`, walrus)	S2, S3, S5

Scaling to the time budget

- **~10 hours:** drop the Pythonic-toolkit tour in S5 Part B and the `pandas` / `json` teaser in S4; keep both halves' core of every session.
- **~12 hours:** run S1–S5 as written, two hours each (recommended).
- **~14 hours:** add the S6 capstone.

Appendix B — Connection Map (education-research bridges)

Connection Map — Python ⇄ Education-Research Background

> From the *personalized-syllabus* method: each new programming concept is bridged to something the student already knows from education research, and — crucially — **where the bridge breaks** is named, so the new concept isn't quietly collapsed into the familiar one. Use these as the "hooks" when introducing each topic; they cut learning time dramatically for an expert adult.

---

1. Variables & assignment *(S1)*

- **Maps onto:** named quantities in a stats model — `n`, `mean`, `alpha`.
- **Where it breaks:** in math, `x = x + 1` is a contradiction; in Python `=` is an *action* (rebind the name), not an assertion of equality. A variable is a **label stuck on an object**, not a box that holds a value. This single reframing prevents most aliasing confusion later.

2. Types & dynamic typing *(S1)*

- **Maps onto:** measurement levels — nominal/ordinal/interval/ratio. Python's `str` / `bool` / `int` / `float` are loosely analogous (categorical vs counted vs continuous).
- **Where it breaks:** SPSS/R columns have *one declared type per column*; a Python variable can hold a string now and an int later. The discipline a column gives you for free, you must supply yourself. This is exactly why the comparison traps in S1 (Part B) exist.

3. `==` vs `is` *(S1)*

- **Maps onto:** the difference between **two questionnaires with identical answers** (equal in value) and **the same physical respondent** (identical individual). Two students can have the same GPA (`==`) without being the same person (`is`).
- **Where it breaks:** the analogy is exact for objects, but Python adds an implementation wrinkle (small-integer caching) that has no analogue in research — flagged as a "do not rely on this."

4. Boolean as a subclass of int (`True == 1`) *(S1)*

- **Maps onto: dummy coding** — you already encode "treatment = 1, control = 0" and then *sum* the dummies to count cases. Python literally does this: `sum([True, False, True]) == 2`.

- **Where it breaks:** in your data the 1/0 is a convention you imposed; in Python it's baked into the language, so `True + True == 2` works even when you didn't intend arithmetic on a flag.

5. Float precision (`0.1 + 0.2 != 0.3`) *(S1)*

- **Maps onto: rounding/measurement error**. You already never test two measured scores for exact equality; you use tolerances and report to 2 decimals.
- **Where it breaks:** here the error isn't in the data — it's in how the *computer* stores decimals in binary. Same instinct (`math.isclose`, round for display), new cause.

6. Lists / dicts / DataFrames *(S2, S4)*

- **Maps onto:** a `list` of `dict`s is a **tidy dataset**: each dict is a *row/respondent*, each key is a *variable/column*. A `dict` alone is one record's variable→value map.
- **Where it breaks:** a spreadsheet enforces rectangularity; a list of dicts does not — rows can have missing or extra keys, which is the source of many bugs (and why we validate in S4).

7. Functions *(S3)*

- **Maps onto:** a **statistical formula or a coding scheme**: defined once, applied to many cases, same rule every time → reproducibility, the thing you care about most as a researcher.
- **Where it breaks:** functions can carry *hidden state* (mutable defaults, globals) so the "same input → same output" promise can silently fail. That trap (S3) is the whole reason to learn scope.

8. Exceptions & validation *(S4)*

- **Maps onto: data cleaning** — out-of-range Likert values, blank cells, "N/A" typed into a numeric field. You already have a mental model of dirty data; exceptions are how code reacts to it.
- **Where it breaks:** cleaning in SPSS is a one-time batch pass; in a program you decide *at runtime*, per value, whether to skip, fix, or stop — a more granular control than a recode syntax file.

9. Regular expressions *(S5)*

- **Maps onto: search-and-filter in a corpus / qualitative coding** — finding every response matching a pattern, extracting IDs, normalizing free-text.
- **Where it breaks:** regex matches *surface form*, not meaning. It's powerful for structure (emails, dates, IDs) and treacherous for semantics — the opposite trade-off from human coding.

10. Classes / OOP *(S5)*

- **Maps onto:** an **operational definition / construct** — a `Student` class bundles the attributes and behaviors you've decided "count" as a student, like an operationalized variable.

- **Where it breaks:** a construct is a measurement choice; a class is also *behavior* (methods) and *enforced rules* (validation in setters), so it's a construct that can defend its own integrity.

11. Recursion *(S3)*

- **Maps onto:** a **hierarchy / nested structure** — coding schemes with sub-codes, threaded discussion data, folder trees of data files, nested JSON survey exports. A procedure "defined in terms of itself" mirrors data that contains smaller copies of itself.
- **Where it breaks:** recursion is elegant only when the structure is genuinely nested and the base case is reachable; for a flat sequence, a loop is clearer, and Python's stack limit (~1000, no tail-call optimization) makes very deep recursion a liability rather than a virtue.

Questions to hold while learning (Socratic spine)

These recur across sessions; the teacher should keep returning to them.

1. *Is this a question about value, or about identity?* (drives `==` vs `is`, copy vs alias)
2. *What type is this right now, and who decided that?* (dynamic typing discipline)
3. *Could this input be dirty? What happens if it is?* (defensive mindset)
4. *Is the same rule guaranteed to run the same way every time?* (reproducibility / pure functions)
5. *Is there a more Pythonic, more readable way to say this?* (the Zen of Python's "readability counts")

Appendix C — Per-Session Quizzes & Answer Keys

Per-Session Quizzes (with answer keys)

Each quiz is ~6 questions (about three per half), ~8 minutes, given at the end of the session. Every question maps to a session objective. **Teacher:** ask the student to *predict* code output before they run it — the surprise is the assessment. Answers are at the end of each session block.

Session 1 — Running Python, Types & the Type Traps

Part A — Variables & Types

1. What type does `input("x: ")` return, always?
2. What does `"5" + "3"` evaluate to? How do you get `8`? And what is `int(3.9)`?

Part B — The Dynamic-Typing Traps

3. `a = [1,2]; b = [1,2]` — is `a == b`? is `a is b`? Why?
4. What is `True + True`? Why?
5. Is `0.1 + 0.2 == 0.3` True or False? How should you compare them?
6. What does `5 == "5"` give? What about `5 > "5"`?

Answers: 1. `str`. 2. `"53"`; use `int("5") + int("3")`; `int(3.9)` is `3` (truncates).

3. `==` True (same value), `is` False (different objects). 4. `2` (bool is a subclass of int).
4. False; use `math.isclose(0.1+0.2, 0.3)` (or round). 6. `False`; `5 > "5"` raises `TypeError` (can't order int vs str).

Session 2 — Control Flow & Data Structures

Part A — Conditionals & Loops

1. Rewrite `if x >= 90 and x < 100:` using a chained comparison.
2. What is `5 and 0`? What is `0 or "hi"`? Why aren't they `True` / `False`?
3. What does `list(range(1, 5))` produce?

Part B — Data Structures

4. Which are mutable: list, tuple, dict, set?
5. `a = [1,2]; b = a; a.append(3)` — what is `b`? Why?
6. (Practical) Write a one-line dict comprehension mapping each name in `names` to `0`.

Answers: 1. `if 90 <= x < 100:`. 2. `0` and `"hi"` — `and` / `or` return an operand, not a bool.

3. `[1, 2, 3, 4]` (5 excluded). 4. list, dict, set are mutable; tuple is not. 5. `[1, 2, 3]` — `b` is an alias for the same list. 6. `{n: 0 for n in names}`.

Session 3 — Functions, Scope & Recursion

Part A — Functions & Scope

1. What's the difference between `return` and `print`?
2. Why is `def f(x, items=[])` dangerous? What's the fix?
3. What does a function with no `return` statement return? Are type hints enforced at runtime?

Part B — Recursion

4. What two parts must every recursive function have?
5. What error comes from a base case that's never reached, and why doesn't Python just keep going?
6. Why is recursion a natural fit for nested data (a list of lists, or nested JSON)?

Answers: 1. `return` hands a value to the caller; `print` only displays. 2. The default list is created once and persists across calls; use `items=None` then create inside. 3. `None`; and no — type hints are documentation (`mypy` checks them optionally). 4. A **base case** (stops) and a **recursive case** (calls itself on a smaller input, toward the base case). 5. `RecursionError` — Python has no tail-call optimization, so each pending call keeps a stack frame until the limit (~1000). 6. The data is *defined in terms of itself* (a list may contain lists), so a function defined in terms of itself mirrors its shape and can reach every level.

Session 4 — Exceptions, Files & Research Data

Part A — Exceptions

1. Which exception does `int("N/A")` raise? Wrap `int(value)` so it returns `None` on failure.
2. Why is a bare `except:` dangerous?
3. When should you use `raise` vs `assert`?

Part B — Files & Data

4. What does opening a file in `"w"` mode do to existing contents? Why prefer `with open(...)`?
5. After `csv.DictReader`, what type is each row?
6. CSV values read from a file are what type — and what must you do with numbers?

Answers: 1. `ValueError`; `try: return int(value) except (ValueError, TypeError): return None`.

2. It catches everything (even Ctrl+C and your own typos) and can hide bugs. 3. `raise` to validate real/untrusted input; `assert` for developer sanity checks (can be disabled). 4. Truncates it to empty immediately; `with` auto-closes even if the code crashes. 5. A `dict` keyed by the header row. 6. Strings; convert with `int()` / `float()`.

Session 5 — Regular Expressions, Modules & OOP

Part A — Regular Expressions

1. In regex, what does `.` match? How do you match a literal dot?
2. Why write regex patterns as raw strings `r"..."`?
3. What does `re.search` return when there's no match, and what must you do before `.group()`?

Part B — Modules & OOP

4. In a class, what is `self`?
5. What happens if you iterate a generator twice?
6. Why doesn't a module's `if __name__ == "__main__":` block run when you `import` it?

Answers: 1. Any character (except newline); use `\.` for a literal dot. 2. So backslashes aren't treated as Python string escapes. 3. It returns `None`; check `if m:` before `m.group()` or you'll hit an `AttributeError`. 4. The current instance ("this particular object"). 5. The second pass is empty — a generator is exhausted after one iteration. 6. On import, `__name__` is the module's name, not `"__main__"`, so the block is skipped.

Scoring guide (formative, not graded)

- **All correct, explained why:** ready for the capstone.
- **Right answer, fuzzy why:** re-do that session's `traps-and-gotchas` rows.
- **Wrong on Session 1 Part B items:** revisit the type traps before continuing — they're load-bearing.